SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

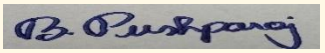
Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	**CSA1008**	**Course Title:**	**Software Engineering**
CO Assessed:	**CO4**	**Assessment:**	**Assessment Tool 2 – CI/CD Pipeline Design Exercise**
Weightage:	**20%**	**Total Marks:**	**25**
CO4 Statement:	**Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics**		
Student Name:	**B. Pushparaj**	**Reg. No.:**	**192421136**
Date:	**27.08.2026**	**Duration:**	**60 minutes**

Code of Conduct

I B. Pushparaj(192421136) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____  _____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, design a CI/CD (Continuous Integration and Continuous Deployment) pipeline for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. Analyze the project requirements and identify the CI/CD needs of the assigned problem statement.
2. Design the CI/CD pipeline workflow showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable tools and technologies for each stage of the pipeline and justify their selection.
4. Define appropriate automated testing strategies to be integrated into the pipeline.

5. Design the deployment strategy, including development, testing/staging, and production environments.
6. Incorporate appropriate security, quality checks, notifications, and rollback mechanisms in the pipeline.
7. Prepare a CI/CD pipeline diagram and explain the workflow from code commit to deployment.

Deliverable: Submit a CI/CD Pipeline Design document containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement**.**

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified

Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation(10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	**Mark**
Requirement & Pipeline Analysis(15%)	**/5**
Pipeline Architecture & Workflow(25%)	**/4**
Tools & Technology Selection(20%)	**/4**
Automated Testing & Quality Integration(15%)	**/4**
Deployment, Security & Rollback Strategy(15%)	**/4**
Pipeline Diagram & Documentation(10%)	**/4**
TOTAL	**/25**

CO4 ASSESSMENT TOOL 2

CI/CD PIPELINE DESIGN EXERCISE

Topic: Accessible Learning Platform

Below is the complete content you can use for your assignment, following the structure of the given assessment.

1. Project Requirement and CI/CD Analysis

Problem Overview

The **Accessible Learning Platform** is a web-based learning system that provides accessible educational resources for students. It supports screen readers, keyboard navigation, voice navigation, audio learning materials, online assessments, progress tracking, and teacher content management.

Since the application contains multiple modules, frequent code changes need to be tested automatically before deployment. A CI/CD pipeline helps developers integrate code, run tests, check quality, build the application, and deploy it safely.

CI/CD Requirements

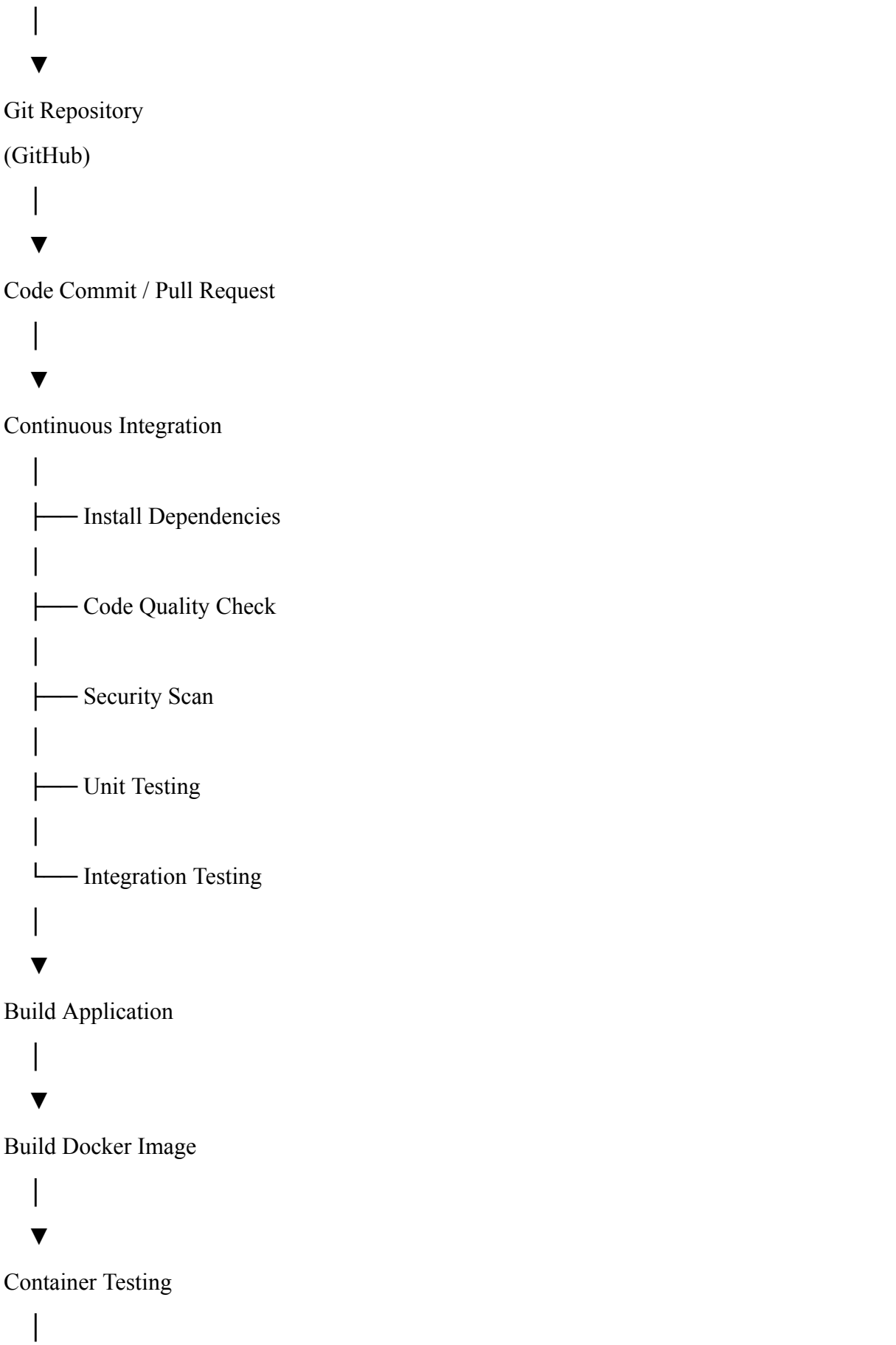
The pipeline should:

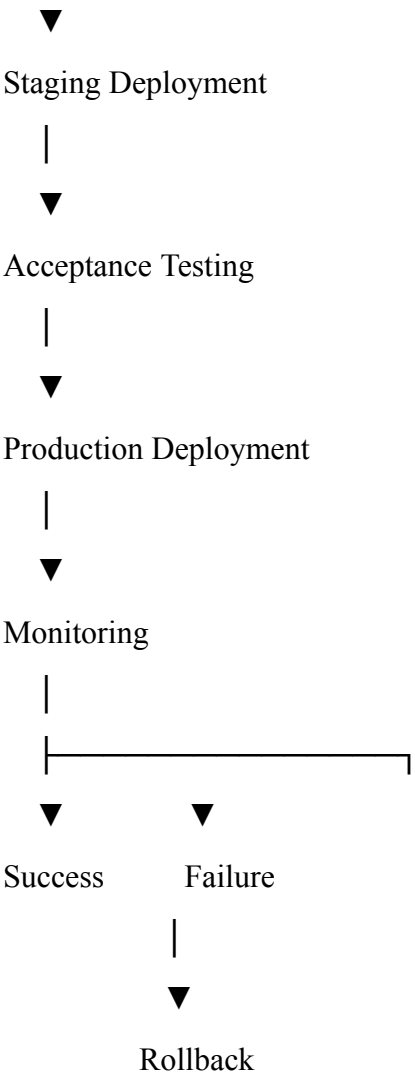
- Automatically start when developers push code to Git.
- Install project dependencies.
- Perform code-quality checks.
- Run automated unit and integration tests.
- Build the application.
- Build the Docker image.
- Test the Docker container.
- Deploy to a staging environment.
- Deploy to production after successful validation.
- Provide notifications when a build or deployment fails.
- Support rollback when a deployment causes problems.

2. Proposed CI/CD Pipeline

Pipeline Workflow

Developer


Git Repository
(GitHub)
Code Commit / Pull Request
Continuous Integration
Install Dependencies
Code Quality Check
Security Scan
Unit Testing
Integration Testing
Build Application
Build Docker Image
Container Testing



3. Tools and Technologies

Pipeline Stage	Tool	Purpose
Source Control	Git	Track source-code changes
Repository	GitHub	Store and collaborate on code
CI/CD	GitHub Actions	Automate the pipeline
Programming	Python / Flask	Develop the application
Testing	PyTest	Run automated tests
API Testing	Postman	Validate APIs
Code Quality	Flake8	Check Python code quality
Security	Bandit	Identify common Python security issues
Containerization	Docker	Package application consistently

Multi-container	Docker Compose	Run application services
Deployment	Docker-based server	Run the application
Monitoring	Logs/health checks	Monitor application status
Notifications	GitHub Actions	Report pipeline results

4. Source Code Management

The project source code is maintained using **Git and GitHub**.

Repository Structure

Accessible-Learning-Platform/

```
│
├── app.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── README.md
│
├── templates/
│   ├── login.html
│   ├── dashboard.html
│   ├── courses.html
│   ├── assessment.html
│   └── progress.html
│
├── static/
│   ├── css/
│   ├── js/
│   └── audio/
│
```

```
├── tests/
│   ├── test_login.py
│   ├── test_courses.py
│   └── test_assessment.py
│
└── .github/
    └── workflows/
        └── ci-cd.yml
```

Git Workflow

Feature Branch

↓

Code Development

↓

Commit

↓

Push to GitHub

↓

Pull Request

↓

Automated CI Tests

↓

Code Review

↓

Merge to Main

5. Continuous Integration

Whenever a developer pushes code to GitHub, GitHub Actions automatically starts the CI pipeline.

CI Stages

Stage 1 – Checkout

The latest source code is downloaded from the repository.

Stage 2 – Install Dependencies

Python dependencies are installed from:

requirements.txt

Stage 3 – Code Quality

The pipeline checks Python source code using Flake8.

Example:

flake8 .

Stage 4 – Security Check

Bandit can scan Python code:

bandit -r .

This helps identify common security problems.

Stage 5 – Automated Testing

PyTest runs the test cases.

pytest

Tests can include:

- Login testing
- Registration testing
- Course testing
- Assessment testing
- Progress testing
- API testing

Stage 6 – Build

After successful testing, the application is prepared for containerization.

6. Docker Integration

The application is containerized using Docker.

Dockerfile

A simple Dockerfile can be:

FROM python:3.11-slim

```
WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Purpose

Instruction	Purpose
FROM	Selects Python base image
WORKDIR	Creates application working directory
COPY	Copies project files
RUN	Installs dependencies
EXPOSE	Indicates application port
CMD	Starts Flask application

7. Docker Compose

Docker Compose can be used to manage the application environment.

Example:

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      - FLASK_ENV=production
```

restart: unless-stopped

The Compose configuration makes application deployment easier and consistent across environments.

8. Automated Testing Strategy

The CI/CD pipeline should perform different levels of testing.

Unit Testing

Tests individual functions.

Example:

Test Login Function

Test Registration Function

Test Score Calculation

Integration Testing

Checks whether different components work together.

Example:

Login → Database → Dashboard

API Testing

Tests Flask API endpoints using automated requests or Postman.

Accessibility Testing

Important for this project.

Check:

- Keyboard navigation
- Screen-reader compatibility
- Button labels
- Form labels
- Audio controls
- Accessible error messages

Regression Testing

Previously working features are tested again whenever new code is added.

9. Deployment Environments

The project uses three environments.

Development

↓

Testing / Staging

↓

Production

Development

Developers create and test new features.

Staging

The complete application is deployed for final testing.

Tests include:

- Functional testing
- Accessibility testing
- Integration testing
- Security testing

Production

After successful staging validation, the application is deployed to the production environment.

10. Deployment Strategy

The deployment process is:

Git Push

↓

Build

↓

Test

↓

Security Check

↓

Docker Build

↓

Staging

↓

Acceptance Testing

↓

Production

Only code that successfully passes the required checks should move to production.

11. Security Integration

Security checks are included before deployment.

Security Measures

- Passwords should not be stored as plain text.
- Authentication should be validated.
- Unauthorized users should not access protected pages.
- Sensitive configuration should be stored using environment variables.
- Dependency vulnerabilities should be checked.
- Docker images should be kept updated.
- Security scanning should run during CI.

Example environment variable:

DATABASE_URL=<secure-value>

SECRET_KEY=<secure-value>

Sensitive values should **not** be committed directly to GitHub.

12. Quality Gates

The pipeline should stop deployment when an important check fails.

Code

↓

Quality Check

↓

FAIL ——————→ Stop Pipeline

|

PASS

↓

Security Check

↓

FAIL ————→ Stop Pipeline

|

PASS

↓

Testing

↓

FAIL ————→ Stop Pipeline

|

PASS

↓

Build

↓

Deploy

This prevents defective code from reaching production.

13. Rollback Strategy

If a newly deployed version has a serious problem, the previous working version can be restored.

Example:

Version 1.0

↓

Version 1.1

↓

Production Error

↓

Rollback

↓

Version 1.0 Restored

Rollback can be triggered when:

- Application does not start.
- Login stops working.
- Assessment results are incorrect.
- Database connection fails.
- Major accessibility functionality breaks.

14. Monitoring and Notification

After deployment, the application should be monitored.

Monitor:

- Application availability
- Server/container status
- Error logs
- Failed requests
- Database connection
- Application health

If the pipeline fails, the developer should receive a notification through the configured GitHub Actions notification mechanism.

15. Sample GitHub Actions Workflow

Create:

.github/workflows/ci-cd.yml

Example:

name: Accessible Learning Platform CI/CD

on:

  push:

    branches:

      - main

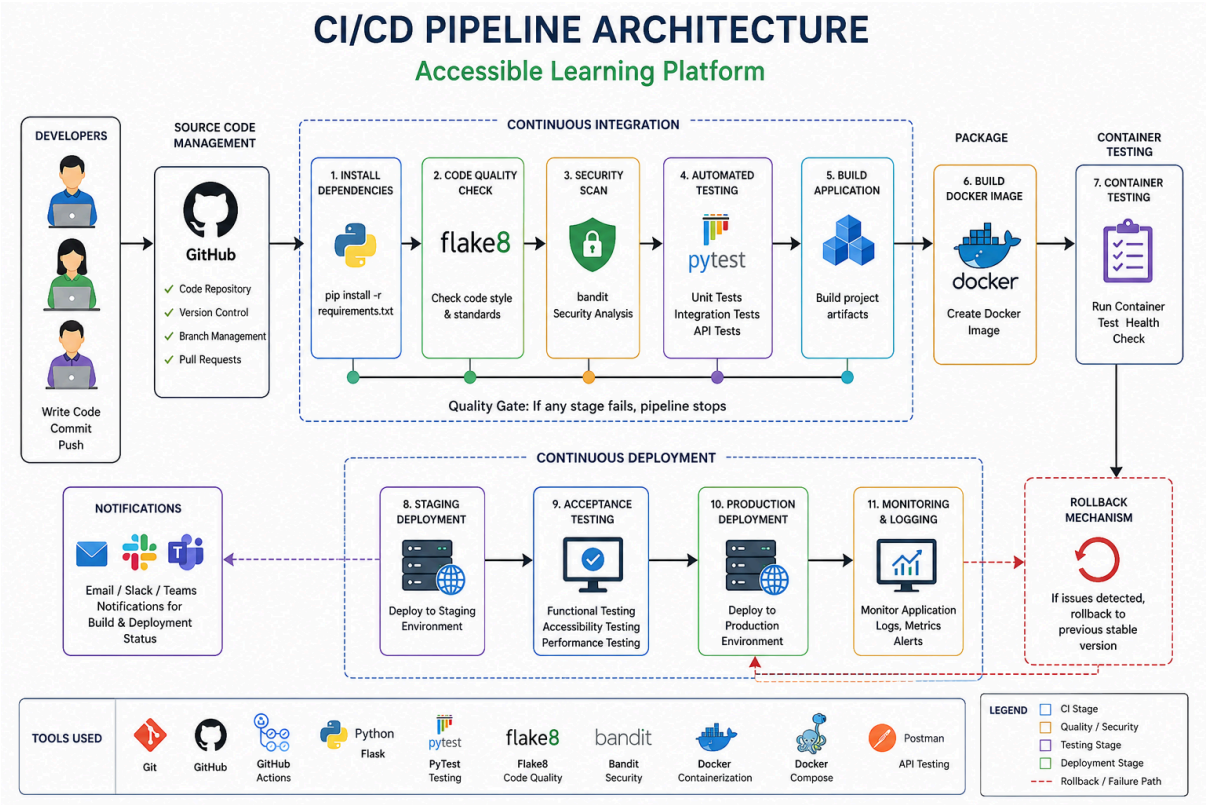
  pull_request:

```
    branches:
      - main

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install Dependencies
        run: |
          pip install -r requirements.txt

      - name: Code Quality Check
        run: |
          flake8 .

      - name: Security Scan
        run: |
          bandit -r .

      - name: Run Tests
        run: |
```

```
      pytest

  docker:
    needs: test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Build Docker Image
        run: |
          docker build -t accessible-learning-platform .

      - name: Test Docker Image
        run: |
          docker images
```

This provides the basic CI portion and Docker build stage. **Actual production deployment commands should be added only after the target hosting environment is selected.**

16. Complete CI/CD Architecture



17. CI/CD Benefits for the Project

Faster Development

Code is automatically tested whenever changes are pushed.

Better Quality

Automated tests detect errors before deployment.

Improved Security

Security checks are included in the development pipeline.

Consistent Deployment

Docker provides the same application environment across systems.

Easy Collaboration

Git and GitHub allow multiple developers to work together.

Reduced Manual Work

Build, test, and deployment processes are automated.

Quick Recovery

Rollback allows the previous stable version to be restored when necessary.

18. Challenges and Future Improvements

Challenges

- Initial GitHub Actions configuration may be difficult.
- Automated accessibility testing requires additional setup.
- Docker configuration may have dependency issues.
- Deployment configuration depends on the selected hosting environment.
- Database configuration must be handled securely.

Future Improvements

- Add automated accessibility testing.
- Add automated performance testing.
- Add Docker image vulnerability scanning.
- Add automated production deployment.
- Add monitoring dashboards.
- Add automated rollback.
- Add notifications for failed deployments.

19. Expected Pipeline Result

The final pipeline should work as:

Developer Push

↓

GitHub

↓

GitHub Actions

↓

Install Dependencies

↓

Code Quality

↓

Security Scan

↓

Automated Tests

↓

Docker Build

↓

Staging

↓

Acceptance Testing

↓

Production

↓

Monitoring

Final Conclusion

The proposed CI/CD pipeline provides an automated process for developing, testing, containerizing, and deploying the **Accessible Learning Platform**. GitHub manages source-code changes, GitHub Actions automates the pipeline, PyTest performs automated testing, Flake8 checks code quality, Bandit performs security analysis, and Docker provides a consistent deployment environment. The inclusion of staging, security checks, monitoring, and rollback improves the **quality, reliability, security, and maintainability** of the system.